Flutter ir Dart aplinkos paruošimas

Diegimo gidas: Flutter SDK, VS Code, Android Studio ir Xcode (Windows, Linux, macOS) bei SimpleStudentMobile paleidimas

Modulis: Heterogeninės sistemos
Grupė: PS-5
2026 m. ruduo
Dėstytojas: Tadas Bulis
Kauno kolegija, Technologijų fakultetas

Turinys

1. Paskirtis
2. Reikalavimai ir pasiruošimas
3. Diegimo eiga – apžvalga
4. 1 žingsnis – būtina programinė įranga
5. 2 žingsnis – Flutter SDK (Dart įeina)
6. 3 žingsnis – VS Code konfigūravimas
7. 4 žingsnis – Android Studio ir Android SDK
8. 5 žingsnis – Android emuliatorius ir tikras įrenginys
9. 6 žingsnis – iOS (tik macOS)
10. 7 žingsnis – patikrinimas su flutter doctor
11. SimpleStudentMobile paleidimas
12. Programėlės sukūrimas telefonams ir planšetėms
13. Problemų sprendimas
14. Komandų atmintinė
15. Šaltiniai

1. Paskirtis

Šis gidas padės paruošti kompiuterį mobiliųjų programėlių kūrimui su **Flutter** (programavimo kalba **Dart**) ir paleisti kurso pavyzdį **SimpleStudentMobile** – mobilųjį SimpleStudentApi klientą Android ir iOS telefonams bei planšetėms.

Flutter – tai Google karkasas, kuriuo iš vieno Dart kodo sukuriamos programėlės Android, iOS, žiniatinkliui ir kompiuteriams. Mūsų heterogeninėje sistemoje tai dar viena technologija, kuri naudoja tą patį REST API kontraktą (swagger.json) kaip .NET ir PHP klientai.

1.1 Ką galima kurti kurioje OS

Jūsų kompiuteris	Android programėlė	iOS programėlė	Pastaba
Windows 10/11 (64 bit)	Taip	Ne	iOS kūrimui Apple reikalauja macOS ir Xcode
Linux (64 bit)	Taip	Ne	Gidas remiasi Ubuntu / Debian
macOS	Taip	Taip	Rekomenduojamas Apple Silicon (M1–M4); Intel Mac palaikymas nutraukiamas

Versijos (2026 m. spalis): Flutter 3.47 (stable kanalas, Dart įeina į Flutter SDK), naujausia Android Studio, Android SDK API 36, naujausias Xcode. Visada diekite naujausią stabilią versiją – komandos ir meniu išlieka tokie patys.

2. Reikalavimai ir pasiruošimas

Kas	Minimaliai	Rekomenduojama
RAM	8 GB	16 GB (Android emuliatorius ir IDE naudoja daug atminties)
Laisva vieta diske	~20 GB (Flutter + Android SDK + emuliatorius)	40 GB+; macOS su Xcode ir simuliatoriais – dar daugiau
Procesorius	64 bitų, su virtualizacija (Intel VT-x / AMD-V)	Virtualizacija įjungta BIOS/UEFI – reikia emuliatoriui
Teisės	Administratoriaus teisės programoms diegti	–
Internetas	Būtinas diegimui (atsisiunčiama keli GB)	Diekite namuose arba kolegijos tinkle, ne mobiliuoju internetu

Svarbu – keliai be tarpų ir lietuviškų raidžių. Flutter, Gradle ir Android įrankiai dažnai neveikia, jei kelyje yra tarpų arba raidžių ą, č, ę, ė, į, š, ų, ū, ž (pvz. `C:\Users\Šarūnas\...`).
Windows: Flutter SDK dėkite į `C:\dev\flutter`, projektus – į `C:\dev\projects`.
macOS / Linux: `~/develop/flutter` ir `~/develop/projects`.

3. Diegimo eiga – apžvalga

Žingsnis	Windows	Linux	macOS
1. Būtina programinė įranga (Git, VS Code)	Taip	Taip	Taip
2. Flutter SDK (su Dart)	Taip	Taip	Taip
3. VS Code konfigūravimas	Taip	Taip	Taip
4. Android Studio ir Android SDK	Taip	Taip	Taip
5. Android emuliatorius / tikras telefonas	Taip	Taip	Taip
6. Xcode, iOS simuliatorius, iPhone/iPad	–	–	Taip
7. Patikrinimas: flutter doctor	Taip	Taip	Taip
8. SimpleStudentMobile paleidimas	Taip	Taip	Taip

Užtenka vieno redaktoriaus: **VS Code** (lengvesnis, rekomenduojamas kodo rašymui) arba **Android Studio**. Tačiau Android Studio reikia bet kuriuo atveju – iš jos diegiamas Android SDK ir emuliatorius.

4. 1 žingsnis – būtina programinė įranga

Windows

1. Atsisiųskite ir įdiekite **Git for Windows**: https://git-scm.com/downloads/win (galima palikti numatytuosius nustatymus).
2. Atsisiųskite ir įdiekite **Visual Studio Code**: https://code.visualstudio.com. Diegimo metu pažymėkite „Add to PATH“ ir „Open with Code“.
3. Atidarykite naują **PowerShell** langą ir patikrinkite:

```
git --version
code --version
```

macOS

1. Terminale įdiekite Xcode komandinės eilutės įrankius (juose yra Git):

```
xcode-select --install
```

2. Atsisiųskite **VS Code** (https://code.visualstudio.com), perkelkite į **Applications**.
3. VS Code atidarykite Command Palette (`Cmd+Shift+P`) → **Shell Command: Install 'code' command in PATH**.

Linux (Ubuntu / Debian)

```
sudo apt-get update -y && sudo apt-get upgrade -y
sudo apt-get install -y curl git unzip xz-utils zip libglu1-mesa

# VS Code (vienas iš būdų):
sudo snap install code --classic
```

Kitose distribucijose tuos pačius paketus įdiekite savo paketų tvarkykle (dnf, pacman ir kt.).

5. 2 žingsnis – Flutter SDK (Dart įeina)

Dart atskirai diegti **nereikia** – jis yra Flutter SDK viduje (`flutter/bin/dart`). Pasirinkite vieną iš dviejų būdų.

5.1 A variantas – per VS Code (rekomenduojama)

1. VS Code → Extensions (`Ctrl+Shift+X`, macOS `Cmd+Shift+X`) → ieškokite **Flutter** (leidėjas Dart Code) → **Install**. Kartu įsidiegs ir **Dart** plėtinys.
2. Command Palette (`Ctrl+Shift+P` / `Cmd+Shift+P`) → įveskite `flutter` → **Flutter: New Project**.
3. Kai VS Code paklaus, kur yra Flutter SDK, spauskite **Download SDK**.
4. Pasirinkite aplanką (Windows `C:\dev`, macOS/Linux `~/develop`) → **Clone Flutter**. Atsisiuntimas užtrunka kelias minutes.
5. Kai pasiūlys – spauskite **Add SDK to PATH**.
6. Uždarykite visus terminalus ir **perkraukite VS Code**. Projekto kūrimo langą galite atšaukti.

5.2 B variantas – rankinis diegimas

Atsisiųskite naujausią **stable** archyvą: https://docs.flutter.dev/install/archive (macOS – pasirinkite Apple Silicon arba Intel).

Windows (PowerShell)

```
New-Item -ItemType Directory -Force C:\dev
Expand-Archive -Path "$env:USERPROFILE\Downloads\flutter_windows_3.47.x-stable.zip" -
DestinationPath C:\dev
# Rezultatas: C:\dev\flutter\bin\flutter.bat
```

PATH: `Win + Pause` (arba Pradžia → „Edit the system environment variables“) → **Advanced system settings** → **Environment Variables…** → **User variables** → **Path** → **Edit** → **New** → `C:\dev\flutter\bin` → **Move Up** į viršų → **OK** ×3. Atidarykite naują terminalą.

> Jei po išarchyvavimo `bin` aplanke nėra `flutter.bat`, jį greičiausiai užblokavo antivirusinė programa – pridėkite `C:\dev\flutter` prie išimčių ir išarchyvuokite iš naujo.

macOS (zsh)

```
mkdir -p ~/develop
unzip ~/Downloads/flutter_macos_arm64_3.47.x-stable.zip -d ~/develop/
echo 'export PATH="$HOME/develop/flutter/bin:$PATH"' >> ~/.zprofile
source ~/.zprofile
```

Linux (bash)

```
mkdir -p ~/develop
tar -xf ~/Downloads/flutter_linux_3.47.x-stable.tar.xz -C ~/develop/
echo 'export PATH="$HOME/develop/flutter/bin:$PATH"' >> ~/.bashrc
source ~/.bashrc
# zsh naudotojams: >> ~/.zshenv
```

5.3 Patikrinimas

```
flutter --version
dart --version
flutter doctor
```

Pirmas `flutter` paleidimas atsisiunčia papildomus įrankius – tai normalu. `flutter doctor` šiame etape dar rodys trūkstamą Android / Xcode – juos įdiegsime toliau. Atnaujinti vėliau: `flutter upgrade`.

6. 3 žingsnis – VS Code konfigūravimas

Jei diegėte B variantu, VS Code dar įdiekite plėtinį **Flutter** (Dart Code) – žr. 5.1 punkto 1 žingsnį. Plėtinys pats suras Flutter SDK per PATH.

6.1 Naudingos komandos ir klavišai

Veiksmas	Windows / Linux	macOS
Command Palette	`Ctrl+Shift+P`	`Cmd+Shift+P`
Flutter: Run Flutter Doctor	Command Palette	Command Palette
Pasirinkti įrenginį (emuliatorių, telefoną)	Status bar apačioje dešinėje arba **Flutter: Select Device**	tas pats
Paleisti su derinimu	`F5`	`F5`
Paleisti be derinimo	`Ctrl+F5`	`Ctrl+F5`
Hot reload (pakeitimai be perkrovimo)	Išsaugoti failą `Ctrl+S`	`Cmd+S`
Hot restart (iš naujo, būsena išvaloma)	`Ctrl+Shift+F5`	`Cmd+Shift+F5`
Gauti paketus (pub get)	Atsidarius pubspec.yaml – automatiškai arba **Flutter: Get Packages**	tas pats
Flutter DevTools	**Flutter: Open DevTools**	tas pats

6.2 Rekomenduojami nustatymai

Command Palette → **Preferences: Open User Settings (JSON)** ir pridėkite:

```
"[dart]": {
  "editor.formatOnSave": true,
  "editor.rulers": [80]
},
"dart.previewFlutterUiGuides": true
```

Projektą atidarykite per **File → Open Folder…** ir pasirinkite patį projekto aplanką (tą, kuriame yra `pubspec.yaml`), o ne visą repozitoriją.

7. 4 žingsnis – Android Studio ir Android SDK

7.1 Android Studio diegimas

Atsisiųskite naujausią stabilią versiją: https://developer.android.com/studio

OS	Kaip įdiegti
Windows	Paleiskite `.exe` diegimo programą, palikite pažymėtą „Android Virtual Device“.
macOS	Atidarykite `.dmg`, nutempkite **Android Studio** į **Applications**. Apple Silicon kompiuteriams rinkitės „Mac with Apple chip“ versiją.
Linux	Išarchyvuokite `.tar.gz` (pvz. į `/opt/android-studio`) ir paleiskite `bin/studio.sh`, arba: `sudo snap install android-studio --classic`. Prieš tai įdiekite 64 bitų sistemai reikalingas 32 bitų bibliotekas – sąrašas: https://developer.android.com/studio/install#64bit-libs

Pirmo paleidimo **Setup Wizard** pasirinkite **Standard** ir leiskite atsisiųsti siūlomus komponentus.

7.2 SDK Manager

Welcome lange **More Actions → SDK Manager** (atidarius projektą: **Tools → SDK Manager**).

- **SDK Platforms** skirtukas: pažymėkite **Android API 36** (pirmą įrašą su API Level 36).
- **SDK Tools** skirtukas: **Android SDK Build-Tools**, **Android SDK Command-line Tools**, **Android Emulator**, **Android SDK Platform-Tools**, **CMake**, **NDK (Side by side)**.
- Spauskite **Apply → OK → Finish**.

> **Android SDK Command-line Tools** dažnai pamirštama. Be jų `flutter doctor` rodo „cmdline-tools component is missing“ ir negalima priimti licencijų.

7.3 Flutter įskiepis Android Studio

1. Windows/Linux: **File → Settings → Plugins**; macOS: **Android Studio → Settings → Plugins**.
2. **Marketplace** skirtuke ieškokite **Flutter → Install** (kartu bus pasiūlytas ir **Dart**) **→ Restart IDE**.
3. Po perkrovimo Welcome lange atsiras **New Flutter Project**. Jei paklaus Flutter SDK kelio – nurodykite `C:\dev\flutter` arba `~/develop/flutter`.

7.4 Android licencijos

```
flutter doctor --android-licenses
# kiekvienai licencijai atsakykite: y
```

Jei Flutter neranda Android SDK, nurodykite kelią: `flutter config --android-sdk <kelias>`. Numatytieji keliai: Windows `%LOCALAPPDATA%\Android\Sdk`, macOS `~/Library/Android/sdk`, Linux `~/Android/Sdk`. Java (JDK) atskirai diegti nereikia – Flutter naudoja Android Studio pridedamą JDK.

8. 5 žingsnis – Android emuliatorius ir tikras įrenginys

8.1 Virtualizacija (emuliatoriaus spartai)

OS	Ką padaryti
Windows	BIOS/UEFI įjunkite **Intel VT-x** arba **AMD-V / SVM**. Tada Pradžia → `optionalfeatures` → pažymėkite **Windows Hypervisor Platform** → perkraukite kompiuterį.
Linux	Reikia KVM: `sudo apt install -y qemu-kvm` ir `sudo adduser $USER kvm`, tada atsijunkite ir vėl prisijunkite. Patikrinimas: `ls -l /dev/kvm`.
macOS	Nieko daryti nereikia. Apple Silicon kompiuteriuose naudokite **ARM** sistemos atvaizdus (Android Studio pasiūlo automatiškai).

8.2 Emuliatoriaus sukūrimas (telefonas ir planšetė)

1. Android Studio: **More Actions** → **Virtual Device Manager** (arba **Tools** → **Device Manager**) → **+ / Create Virtual Device**.
2. **Phone** kategorijoje pasirinkite pvz. **Pixel 9** → **Next**.
3. Pasirinkite sistemos atvaizdą su **API 36** (x86_64 – Intel/AMD kompiuteriams, ARM – Apple Silicon) → atsisiųskite → **Next**.
4. **Additional settings** → **Emulated Performance** → **Graphics acceleration**: pasirinkite variantą su **Hardware** → **Finish**.
5. Pakartokite su **Tablet** kategorija (pvz. **Pixel Tablet**) – patikrinsite, kaip programėlė atrodo planšetėje.
6. Paleiskite emuliatorių ▶️ mygtuku Device Manager lange.

```
flutter emulators                # emuliatorių sąrašas
flutter emulators --launch <id>  # paleisti
flutter devices                  # įrenginys turi būti sąraše
```

8.3 Tikras Android telefonas ar planšetė

1. Telefone: **Settings** → **About phone** → 7 kartus bakstelėkite **Build number** – įsijungs **Developer options**.
2. **Settings** → **System** → **Developer options** → įjunkite **USB debugging** (pasirinktinai – **Wireless debugging**).
3. **Windows:** įdiekite gamintojo USB tvarkyklę (https://developer.android.com/studio/run/oem-usb). macOS ir Linux tvarkyklių nereikia.
4. Prijunkite USB kabeliu ir telefone patvirtinkite „Allow USB debugging?“.
5. Patikrinkite: `flutter devices`.

9. 6 žingsnis – iOS (tik macOS)

9.1 Xcode

1. Įdiekite naujausią **Xcode** iš Mac App Store arba https://developer.apple.com/xcode/ (užima daug vietos, diegimas ilgas).
2. Terminale sukonfigūruokite įrankius, priimkite licenciją ir atsisiųskite iOS platformą su simuliatoriais:

```
sudo sh -c 'xcode-select -s /Applications/Xcode.app/Contents/Developer && xcodebuild -
runFirstLaunch'
sudo xcodebuild -license
xcodebuild -downloadPlatform iOS
```

9.2 CocoaPods ir Swift Package Manager

Flutter 3.47 iOS priklausomybėms pagal nutylėjimą naudoja **Swift Package Manager** (atskirai diegti nereikia). Tačiau `flutter doctor` vis tiek rodo **„× CocoaPods not installed“**, nes CocoaPods reikia įskiepiams, kurie SPM dar nepalaiko. Įdiekite jį – užtruks kelias minutes.

1) Homebrew (jei dar neturite)

Patikrinkite: `brew --version`. Jei komanda nerasta – įdiekite Homebrew (https://brew.sh), terminale įklijuokite:

```
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Diegimo programa paprašys Mac slaptažodžio (rašant nesimato – tai normalu) ir **Enter** patvirtinimo. **Apple Silicon (M1–M4)** kompiuteryje pabaigoje ji parodo „Next steps“ – įvykdykite jas, kad `brew` būtų PATH:

```
echo 'eval "$(/opt/homebrew/bin/brew shellenv)"' >> ~/.zprofile
eval "$(/opt/homebrew/bin/brew shellenv)"
brew --version          # turi parodyti versiją
```

Intel Mac kompiuteryje Homebrew diegiamas į `/usr/local` ir PATH nustatyti nereikia.

2) CocoaPods

```
brew install cocoapods
pod --version           # pvz. 1.16.x
flutter doctor          # Xcode eilutė turi būti [✓]
```

Daugiau nieko daryti nereikia: `pod install` Flutter paleidžia pats per `flutter run` / `flutter build ios`, kai projekte yra įskiepių, naudojančių CocoaPods.

> Rekomenduojamas būdas – **Homebrew**. `sudo gem install cocoapods` su macOS sistemine Ruby dažnai baigiasi klaidomis (per sena Ruby versija, teisių problemos), todėl jo nenaudokite, nebent žinote, ką darote (pvz. naudojate rbenv / rvm).

Dažniausios klaidos

Klaida	Sprendimas
`brew: command not found` (po Homebrew diegimo)	Neįvykdyti „Next steps“ – paleiskite dvi `brew shellenv` eilutes iš 1) žingsnio ir atidarykite naują terminalą.
`pod: command not found` arba doctor vis dar rodo ×	Uždarykite terminalą ir VS Code / Android Studio, atidarykite iš naujo. Patikrinkite `which pod` (turi būti `/opt/homebrew/bin/pod`).
„CocoaPods installed but not working“	`brew reinstall cocoapods`. Jei anksčiau buvo diegta per gem: `sudo gem uninstall cocoapods`, tada `brew link --overwrite cocoapods`.
„CocoaPods's specs repository is too out-of-date“	`pod repo update`, tada `flutter clean` ir `flutter run`.
Klaidos `pod install` metu konkrečiame projekte	`cd ios && rm -rf Pods Podfile.lock && cd ..`, tada `flutter clean`, `flutter pub get`, `flutter run`.

9.3 iOS simuliatorius (iPhone ir iPad)

```
open -a Simulator          # Xcode 26 ir senesni
open -a DeviceHub          # Xcode 27 (Simulator pervadintas)
flutter emulators --launch apple_ios_simulator
```

Kitą įrenginį (pvz. **iPad**) pasirinkite simuliatoriaus meniu **File → Open Simulator**. iPad palaikomas automatiškai – Flutter iOS projektas sukuriamas su `TARGETED_DEVICE_FAMILY = "1,2"` (iPhone + iPad).

9.4 Tikras iPhone ar iPad

1. Prijunkite USB kabeliu, įrenginyje spauskite **Trust**.
2. Įrenginyje: **Settings → Privacy & Security → Developer Mode → On** → perkraukite → **Turn On**. Jei punkto nėra – pirma paleiskite programėlę per Xcode arba `flutter run`.
3. Atidarykite `ios/Runner.xcworkspace` Xcode programoje → **Runner → Signing & Capabilities → Team**: pasirinkite savo Apple ID (užtenka nemokamos paskyros: **Xcode → Settings → Accounts → +**).
4. **Bundle Identifier** pakeiskite į unikalų, pvz. `lt.kaunokolegija.simpleStudentMobile.vardenis`.
5. Pirmą kartą įrenginyje: **Settings → General → VPN & Device Management** → jūsų sertifikatas → **Trust**.

> Su nemokama Apple paskyra įdiegta programėlė veikia **7 dienas**, po to ją reikia paleisti iš naujo per Xcode / `flutter run`. App Store publikavimui reikia mokamos Apple Developer Program narystės.

10. 7 žingsnis – patikrinimas su flutter doctor

```
flutter doctor

Doctor summary (to see all details, run flutter doctor -v):
[✓] Flutter (Channel stable, 3.47.x, ...)
[✓] Android toolchain - develop for Android devices (Android SDK version 36.x)
[✓] Xcode - develop for iOS and macOS (Xcode 2x.x)        <- tik macOS
[✓] Chrome - develop for the web
[✓] Android Studio (version 202x.x)
[✓] VS Code (version 1.1xx)
[✓] Connected device (2 available)
[✓] Network resources

• No issues found!
```

Eilutė	Ar būtina šiam kursui?
Flutter, Android toolchain, Android Studio	**Taip** – visose OS
Xcode	**Taip** tik macOS, jei kursite iOS versiją
VS Code	Jei juo naudojatės
Chrome	Ne (reikalinga tik žiniatinkliui) – galima ignoruoti
Visual Studio (Windows)	Ne – reikalinga tik Windows darbalaukio programoms. Nediekite.
Linux toolchain (Linux)	Ne – reikalinga tik Linux darbalaukio programoms

Detalesnė informacija ir siūlomi sprendimai: `flutter doctor -v`. Visi Android/Xcode punktai turi būti pažymėti [✓] prieš pereinant prie kito skyriaus.

11. SimpleStudentMobile paleidimas

Projektas yra `code-examples/student-docker-lab-plus-mssql/SimpleStudentMobile` (kurso GitHub repozitorijoje – kataloge `student-docker-lab-plus-mssql`). Jame du skirtukai: **Home** (projekto informacija, API ryšio patikrinimas) ir **Students** (CRUD: sąrašas, kūrimas, redagavimas, trynimas).

11.1 Paleiskite API

```
cd student-docker-lab-plus-mssql
docker-compose -f docker-compose.infra.yml up -d   # SQL Server (vieną kartą)
docker-compose up --build -d                        # .NET API :6001, PHP API :6002
# patikrinimas naršyklėje: http://localhost:6001/swagger
```

XAMPP naudotojams: vietoje Docker galite naudoti PHP API iš `SimpleStudentPhpApi` (`htdocs/simpleStudentApi`, MySQL) – adresas nurodytas 11.4 lentelėje.

11.2 Paruoškite Flutter projektą

```
cd SimpleStudentMobile
flutter pub get
```

Projekte jau yra paruošti `android/` ir `ios/` aplankai, kuriuose leista kreiptis į API per **HTTP**:

- `android/app/src/main/AndroidManifest.xml` – `INTERNET` leidimas ir `android:usesCleartextTraffic="true"`;

- `ios/Runner/Info.plist` – `NSAppTransportSecurity` (`NSAllowsArbitraryLoads`) ir `NSLocalNetworkUsageDescription`.

> **Jei `android/` ar `ios/` aplanko nėra** (ištrynėte, kopijavote tik `lib/`) arba trūksta HTTP leidimų – paleiskite `dart run tool/configure_platforms.dart`. Jis sugeneruoja trūkstamus aplankus per `flutter create` (esamų failų neperrašo) ir pridės HTTP leidimus. Paleisti galima kiek norite kartų.

> Android 9+ ir iOS pagal nutylėjimą blokuoja nešifruotą HTTP. Leidžiame tik todėl, kad tai laboratorinis darbas – realioje programėlėje naudojamas HTTPS.

11.3 Paleiskite programėlę

- **VS Code:** File → Open Folder → `SimpleStudentMobile` → apačioje pasirinkite įrenginį → `F5`.
- **Android Studio:** File → Open → `SimpleStudentMobile` → viršuje įrenginių sąraše pasirinkite emuliatorių → ▶️ **Run**.
- **Terminalas:** `flutter run` (jei įrenginių keli: `flutter run -d <id>`).

11.4 API adresas

Telefonas yra **kitas kompiuteris**, todėl `localhost` ne visada reiškia jūsų PC. Programėlė numatytąjį adresą parenka pati, o pakeisti jį galima **Home** skirtuke → **Change** (išsaugoma įrenginyje).

Kur veikia programėlė	API adresas
Android emuliatorius	`http://10.0.2.2:6001` (10.0.2.2 = jūsų kompiuteris)
iOS simuliatorius (Mac)	`http://localhost:6001`
Tikras Android per USB	`http://localhost:6001` + `adb reverse tcp:6001 tcp:6001` (11.6 sk.)
Tikras telefonas / planšetė per Wi-Fi	`http://<kompiuterio IP>:6001` – tame pačiame Wi-Fi tinkle (11.6 sk.)
PHP API Docker konteineryje	tas pats, tik portas `6002`
PHP API XAMPP	`http://10.0.2.2/simpleStudentApi` (emuliatorius), `http://localhost/simpleStudentApi` (iOS sim.); jei XAMPP Apache veikia per 8080 – pridėkite `:8080`

Adresą galima nurodyti ir paleidžiant: `flutter run --dart-define=API_BASE_URL=http://192.168.1.20:6001`.

Kompiuterio IP: Windows `ipconfig` (IPv4 Address), macOS `ipconfig getifaddr en0`, Linux `hostname -I`.

Ugniasienė (tik tikram įrenginiui)

```
# Windows (PowerShell kaip administratorius):
New-NetFirewallRule -DisplayName "SimpleStudentApi 6001" -Direction Inbound -Protocol TCP -LocalPort 6001 -Action Allow

# Linux (ufw):
sudo ufw allow 6001/tcp

# macOS: System Settings -> Network -> Firewall (leiskite Docker / com.docker.backend)
```

11.5 Ką išbandyti

1. **Home** → **Test connection**: turi rodyti „SimpleStudentApi v1.2 – ok“ (duomenys iš `GET /api/info`).
2. **Students**: sąrašas (`GET /api/students`), traukite žemyn – atnaujinimas.

3. **+ Add** → užpildykite formą → **Create student** (`POST`). Įveskite neteisingą el. paštą – klaida rodoma po lauku.
4. Bakstelėkite studentą → pakeiskite → **Save changes** (`PUT`). Šiukšliadėžės ikona – trynimas (`DELETE`).
5. Sąraše braukite studentą į kairę – trynimas su patvirtinimu.
6. Pasukite emuliatorių horizontaliai arba paleiskite planšetėje – apatinis meniu virsta šoniniu (**NavigationRail**).
7. Atidarykite tą patį sąrašą naršyklėje (http://localhost:6011 arba :6021) – matysite tuos pačius duomenis: trys klientai, vienas API.
8. Pakeiskite tekstą `lib/screens/home_screen.dart`, išsaugokite – **hot reload** atnaujins ekraną per sekundę.

11.6 Programėlė nemato API (Test connection raudonas)

Dažniausia priežastis – netinkamas API adresas **jūsų aplinkai**. Telefonas ar planšetė yra atskiras kompiuteris, todėl jame `localhost` reiškia patį telefoną, o `10.0.2.2` veikia **tik Android emuliatoriuje** (tai emuliatoriaus slapyvardis kompiuteriui, kuriame jis veikia). Tikrame įrenginyje ir kompiuterio naršyklėje `10.0.2.2` niekur neveda.

> Tipinė klaida tikrame įrenginyje: „The API did not answer within 10 s … Is the address http://10.0.2.2:6001 correct?“ – reiškia, kad programėlė naudoja emuliatoriaus adresą. Pasirinkite adresą iš lentelės žemiau.

Kokį adresą naudoti

Jūsų aplinka	Adresas programėlėje (Home → Change)	Ką reikia padaryti
Android emuliatorius	`http://10.0.2.2:6001`	Nieko – tai numatytasis adresas
iOS simuliatorius (Mac)	`http://localhost:6001`	Nieko – tai numatytasis adresas
Tikras Android, prijungtas **USB** kabeliu (rekomenduojama auditorijoje)	`http://localhost:6001`	`adb reverse tcp:6001 tcp:6001` (žr. A būdą)
Tikras Android ar iPhone/iPad per **Wi-Fi**	`http://<kompiuterio IP>:6001`	Tas pats Wi-Fi tinklas, leista ugniasienėje (žr. B būdą)

A būdas – USB ir adb reverse (tik Android)

`adb reverse` nukreipia telefono `localhost:6001` per USB kabelį į jūsų kompiuterio 6001 portą. Wi-Fi ir ugniasienė nesvarbūs, todėl tai patikimiausias būdas kolegijos tinkle.

```
adb devices                          # telefonas turi būti sąraše su būsena "device"
adb reverse tcp:6001 tcp:6001        # atsako: 6001
adb reverse --list                   # patikrinimas

# jei prijungti keli įrenginiai (pvz. dar veikia emuliatorius):
adb -s <įrenginio ID> reverse tcp:6001 tcp:6001
# įrenginio ID matosi: flutter devices  (pvz. R9TX60KABKK)
```

Tada programėlėje **Home** → **Change** → `http://localhost:6001` → **Test connection**. Arba iškart paleidžiant:

```
flutter run -d <įrenginio ID> --dart-define=API_BASE_URL=http://localhost:6001
```

Jei komanda `adb` nerasta – naudokite pilną kelią:

OS	Kelias iki adb
Windows	`%LOCALAPPDATA%\Android\Sdk\platform-tools\adb.exe`
macOS	`~/Library/Android/sdk/platform-tools/adb`
Linux	`~/Android/Sdk/platform-tools/adb`

> `adb reverse` galioja tol, kol įrenginys prijungtas. Atjungus kabelį, perkrovus telefoną ar adb – komandą paleiskite iš naujo. iPhone/iPad šio būdo neturi – jiems naudokite B būdą.

B būdas – Wi-Fi ir kompiuterio IP adresas

1. Sužinokite kompiuterio IP adresą Wi-Fi tinkle: Windows `ipconfig` (eilutė **IPv4 Address** prie Wi-Fi adapterio), macOS `ipconfig getifaddr en0`, Linux `hostname -I` (pirmas adresas). Pvz. `192.168.1.29`.
2. Telefono **naršyklėje** atidarykite `http://192.168.1.29:6001/swagger`. Jei atsidaro – tinklas tvarkoje.
3. Programėlėje **Home** → **Change** → `http://192.168.1.29:6001` → **Test connection**.

Jei 2 žingsnyje puslapis neatsidaro – problema ne programėlėje, o tinkle:

- **Ugniasienė** blokuoja 6001 portą – leiskite jį (komandos 11.4 skyriuje; macOS: System Settings → Network → Firewall → Options → leiskite Docker / `com.docker.backend`).
- **Wi-Fi izoliuoja įrenginius** (dažna svečių ir viešuose tinkluose, kartais ir kolegijos) – naudokite A būdą (USB) arba telefono „Hotspot“, prie kurio prijunkite kompiuterį.
- **Kitas tinklas** – telefonas mobiliajame internete arba kitame Wi-Fi.

> Kompiuterio IP gali pasikeisti persijungus prie Wi-Fi – jei ryšys vėliau dingo, patikrinkite IP iš naujo.

Patikrinimo eiga (bet kuriai aplinkai)

1. Kompiuterio naršyklėje veikia `http://localhost:6001/swagger`? Jei ne – neveikia API: `docker ps`, `docker-compose up --build -d` (11.1).
2. Programėlėje nustatytas adresas tinka jūsų aplinkai (lentelė aukščiau)?
3. Tikrame įrenginyje: tą patį adresą + `/swagger` atidarykite įrenginio naršyklėje – ar atsidaro?
4. Adresas pakeistas per **Change**? Jis turi pirmenybę prieš `--dart-define`. Grąžinti numatytąjį – **Change** → **Use default**.
5. Klaida „Cleartext HTTP traffic not permitted“ – paleiskite `dart run tool/configure_platforms.dart` ir programėlę iš naujo (ne hot reload).

12. Programėlės sukūrimas telefonams ir planšetėms

Tikslas	Komanda	Rezultatas
Android APK (diegti į bet kurį telefoną / planšetę)	`flutter build apk --release`	`build/app/outputs/flutter-apk/app-release.apk`
Android App Bundle (Google Play)	`flutter build appbundle`	`build/app/outputs/bundle/release/app-release.aab`
Įdiegti į prijungtą įrenginį	`flutter install`	Įdiegia paskutinį sukurtą APK
iOS (tik macOS + Xcode)	`flutter build ios --release`	Toliau Xcode: **Product → Archive**
Įdiegti į iPhone/iPad, kad atsidarytų iš pradžios ekrano	`flutter run --release -d <ID>`	Release versija lieka įrenginyje; terminale `q` – baigti

Laboratoriniam darbui pakanka APK – jį galima nusiųsti ir įdiegti telefone (reikia leisti „Install unknown apps“). Release APK pasirašomas debug raktu, todėl Google Play netinka; tam reikia atskiro pasirašymo rakto.

12.1 iOS: programėlė neatsidaro iš pradžios ekrano

Paleidus **debug** versiją iš pradžios ekrano, iPhone rodo: „In iOS 14+, debug mode Flutter apps can only be launched from Flutter tooling, IDEs with Flutter plugins or from Xcode.“ Tai iOS apribojimas: debug versija (JIT, hot reload) veikia tik prijungus derinimo įrankį. Projekto nustatymais to pakeisti negalima.

Sprendimas – įdiegti **release** (arba profile) versiją. Ji atsidaro kaip įprasta programėlė:

```
flutter devices
flutter run --release -d <iPhone ID> --dart-define=API_BASE_URL=http://<kompiuterio IP>:6001
```

- Įdiegus terminale galima spausti `q` – programėlė lieka iPhone ir atsidaro iš pradžios ekrano.
- Release versijoje nėra hot reload – kūrimui naudokite įprastą `flutter run` (debug).
- Su nemokama Apple paskyra programėlė veikia 7 dienas – po to įdiekite iš naujo ta pačia komanda.
- Per Xcode: **Product → Scheme → Edit Scheme… → Run → Build Configuration → Release**, tada **Run**.
- Android šio apribojimo neturi – debug APK atsidaro iš pradžios ekrano.

13. Problemų sprendimas

Problema	Sprendimas
`flutter` / `dart` is not recognized / command not found	PATH nenurodytas arba terminalas senas. Patikrinkite PATH, uždarykite VISUS terminalus ir VS Code, atidarykite iš naujo.
`cmdline-tools component is missing`	SDK Manager → SDK Tools → pažymėkite **Android SDK Command-line Tools** → Apply.
`Android license status unknown`	`flutter doctor --android-licenses`, atsakyti `y`.
`Unable to locate Android SDK`	`flutter config --android-sdk <kelias iki Sdk>`
Gradle / Java versijos klaidos	Atnaujinkite Android Studio. Jei reikia: `flutter config --jdk-dir <Android Studio>/jbr` (macOS: `.../Android Studio.app/Contents/jbr/Contents/Home`).

Problema	Sprendimas
Gradle klaidos su keistais simboliais kelyje	Projektą ir SDK perkelkite į kelią be tarpų ir lietuviškų raidžių (`C:\dev\...`).
Emuliatorius labai lėtas / nepasileidžia	Įjunkite virtualizaciją (8.1 sk.), Graphics – Hardware, uždarykite kitas sunkias programas. Linux – patikrinkite `/dev/kvm` teises.
`Cannot connect to http://...`	Ar veikia API (http://localhost:6001/swagger)? Ar teisingas adresas jūsų įrenginiui (11.4)? Ugniasienė? Žr. **11.6 sk.**
Tikrame įrenginyje: „... Is the address http://10.0.2.2:6001 correct?“	`10.0.2.2` veikia tik emuliatoriuje. USB: `adb reverse tcp:6001 tcp:6001` ir adresas `http://localhost:6001`; Wi-Fi: `http://<kompiuterio IP>:6001` (11.6 sk.).
`Cleartext HTTP traffic not permitted` (Android)	Paleiskite `dart run tool/configure_platforms.dart` ir programą paleiskite iš naujo (ne hot reload).
`App Transport Security has blocked a cleartext HTTP` (iOS)	Tas pats skriptas; patikrinkite `ios/Runner/Info.plist`.
iPhone: „In iOS 14+, debug mode Flutter apps can only be launched…“	Įdiekite release versiją: `flutter run --release -d <ID>` (12.1 sk.).
Xcode: `could not find included file 'Generated.xcconfig'`	Pirma paleiskite `flutter pub get` projekto aplanke, tada atidarykite `ios/Runner.xcworkspace`.
`CocoaPods not installed` (macOS)	Įdiekite pagal 9.2 sk. (Homebrew → `brew install cocoapods`), tada `flutter doctor`.
iOS: „No valid code signing certificates“	Xcode → Runner → Signing & Capabilities → pasirinkite Team, pakeiskite Bundle Identifier.
Keisti paketų / build klaidos po atnaujinimo	`flutter clean`, tada `flutter pub get` ir paleisti iš naujo.
Windows: dingo `flutter.bat`	Antivirusinė karantine – pridėkite `C:\dev\flutter` prie išimčių, išarchyvuokite iš naujo.

14. Komandų atmintinė

Komanda	Paskirtis
`flutter doctor` / `flutter doctor -v`	Aplinkos patikrinimas
`flutter upgrade`	Atnaujinti Flutter ir Dart
`flutter create <vardas>`	Naujas projektas (vardas – mažosiomis raidėmis su _)
`flutter pub get` / `flutter pub add http`	Gauti paketus / pridėti paketą
`flutter devices` / `flutter emulators`	Prijungti įrenginiai / emuliatoriai
`flutter run` / `flutter run -d <id>`	Paleisti (terminale: `r` – hot reload, `R` – hot restart, `q` – baigti)
`flutter analyze`	Statinė kodo analizė
`flutter test`	Testai
`dart format .`	Kodo formatavimas
`flutter clean`	Išvalyti build katalogą
`flutter build apk` / `flutter build ios`	Sukurti programėlę

15. Šaltiniai

- Flutter diegimas su VS Code: https://docs.flutter.dev/install/quick
- Flutter rankinis diegimas: https://docs.flutter.dev/install/manual
- Android nustatymas Flutter: https://docs.flutter.dev/platform-integration/android/setup
- iOS nustatymas Flutter: https://docs.flutter.dev/platform-integration/ios/setup
- Flutter SDK archyvas: https://docs.flutter.dev/install/archive
- Android Studio: https://developer.android.com/studio
- Dart kalba: https://dart.dev/language
- http paketas: https://pub.dev/packages/http